

Bluetooth Hacking 101

Offensive edition

Introducción al curso

Expectativas

- * El estándar Bluetooth es demasiado denso y largo para cubrirlo aquí...
- * Espero enseñaros los fundamentos de la tecnología
- * Espero enseñaros dónde buscar la información que necesitéis

Expectativas

- * La tecnología contiene demasiados puntos a estudiar desde el punto de vista de la seguridad...
- * Espero ver algunos de los casos más relevantes
- * Espero dar los medios para afrontar casos que no hemos visto

Introducción a Bluetooth

Especificación

Bluetooth Special Interest Group (SIG) specifications:

- * <https://www.bluetooth.com/specifications/specs/>

Bluetooth Core Specification:

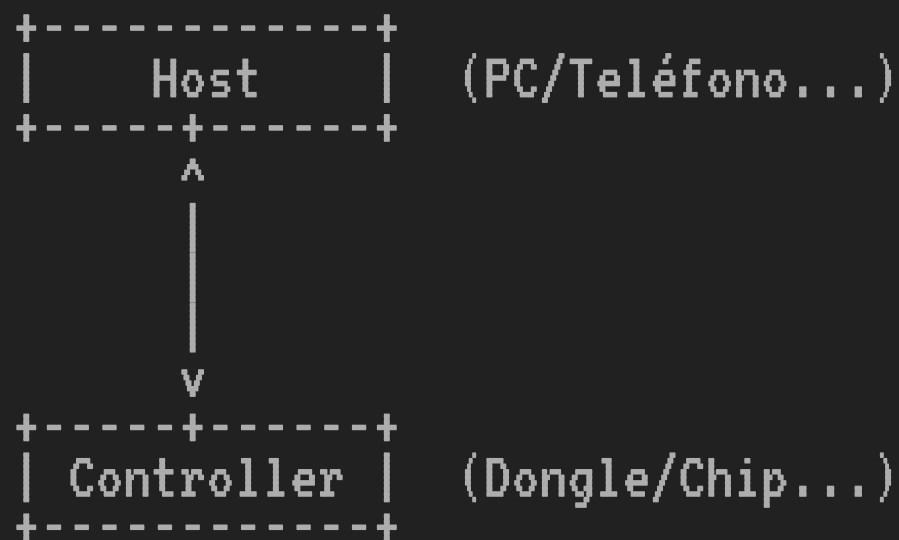
- * <https://www.bluetooth.com/specifications/specs/core-specification-6-2/>

Historia

Año	Versión	Notas
1999	1.0	First Bluetooth specification
2004	2.x + EDR	3 Mbps (Enhanced Data Rate), Secure Simple Pairing (SSP)
2009	3.0 + HS	High Speed via Wi-Fi (up to 24 Mbps)
2010	4.x	BLE, LE Data Length Extension, Privacy 1.2, 6LoWPAN
2016	5.0	2x speed, 4x range, 8x broadcasting, Mesh
2019	5.2	LE Audio, LC3 codec, Isochronous Channels, EATT
2023	6.0	Channel Sounding (cm-level distance), Isochronous 2.0, Monitoring
2025	6.1	Auracast improvements, LE Long Range 2, enhanced Mesh security

Arquitectura de Bluetooth

Arquitectura



Arquitectura

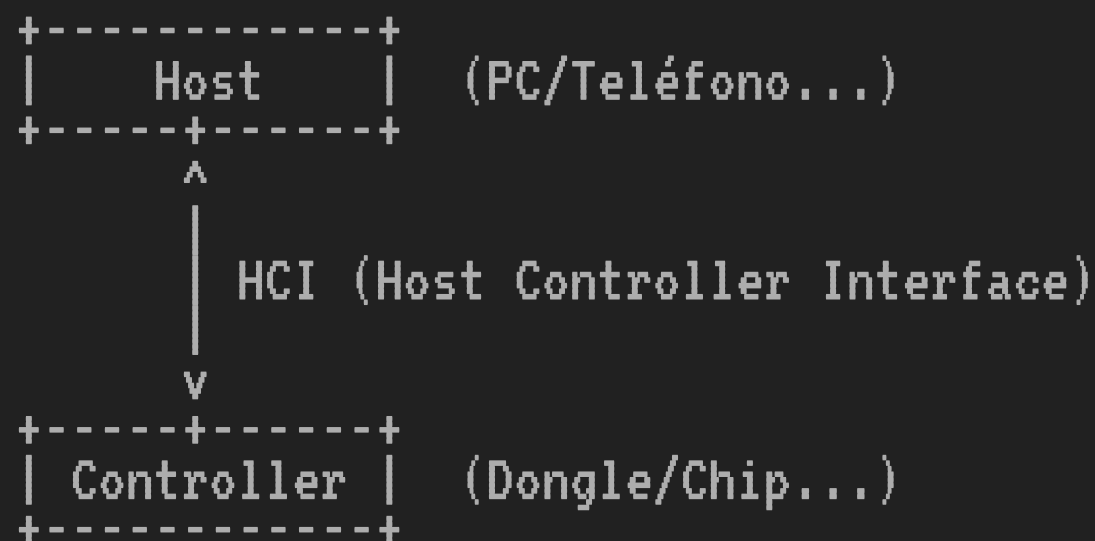
Host:

- * Software que implementa las capas superiores del estándar
- * No requiere hardware específico
- * Puede usar controllers de manera “genérica”

Controller:

- * Sw y hw que implementa las capas inferiores del estándar
- * Este software está muy ligado al hardware en el que corre
- * Hardware especializado para operaciones en tiempo real

Arquitectura



Arquitectura

HCI:

- * Es una especificación de interfaz + protocolo
- * Abstrae el Host del Controller
- * Puede ser Virtual, sobre USB, sobre UART, SD...

Ejercicio 1

- * Conectar nuestro dongle
- * Verificar que el equipo detecta el dongle
- * Identificar nuestro dispositivo (`lsusb`, `ls /dev/tty*`, administrador de dispositivos...)
- * Que transporte usa? USB o UART?

Ejercicio 2

- * Para los que tengan dongle con transporte USB, vamos a ver el descriptor USB de nuestro dispositivo
- * ¿Qué “class of device”, “subclass” y “protocol” tiene nuestro dispositivo? ([Core 6.2, Volume 4, Part B USB Transport Layer](#))

Ejercicio 3

- * En cualquier OS, instalar Python
- * (Opcional) Creamos un virtual environment
- * Instalar el paquete “usbbluetooth”
- * Ejecutar el siguiente script:

```
import usbbluetooth

devices = usbbluetooth.list_controllers()

for dev in devices:
    print(dev)
```

Ejercicio 4

* Hacer una función que nos retorne el controller USB que queremos

```
import usbbluetooth
```

```
def get_controller():
```

```
    devices = usbbluetooth.list_controllers()
```

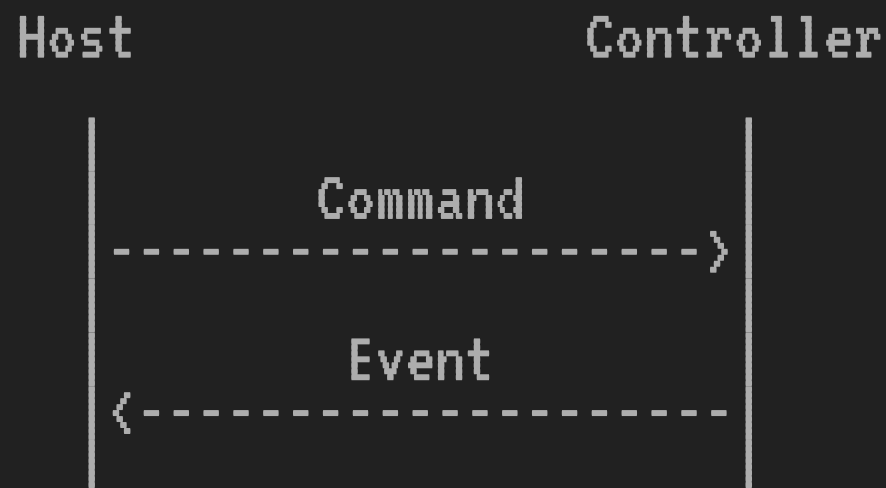
```
    return devices[0]
```

```
c = get_controller()
```

```
print(c)
```


Protocolo HCI

Diagrama de comunicaciones



Tipos de paquete

Command:

- * Del Host al Controller
- * Solicitan datos, inician acciones...

Event:

- * Del Controller al Host
- * Pueden ser en respuesta a un Command o espúreos

Ejercicio 5

- * Construir a mano un paquete de HCI, más concretamente el paquete de reset
- * Core 6.2, Volume 4, Part E, Section 5.4.1

Ejercicio 5

* Construir a mano un paquete de HCI, más concretamente el paquete de reset

* Core 6.2, Volume 4, Part E, Section 5.4.1

HCI Packet Type (Command)

|
v
\x01

Ejercicio 5

* Construir a mano un paquete de HCI, más concretamente el paquete de reset

* Core 6.2, Volume 4, Part E, Section 5.4.1

Opcode (Reset)

|

↓

\x01\x??\x??

Ejercicio 5

* Construir a mano un paquete de HCI, más concretamente el paquete de reset

* Core 6.2, Volume 4, Part E, Section 5.4.1

Opcode (Reset OGF=3 OCF=3)

|

↓

\x01\x03\x0c

Ejercicio 5

* Construir a mano un paquete de HCI, más concretamente el paquete de reset

* Core 6.2, Volume 4, Part E, Section 5.4.1

Parameter length

|

v

\x01\x03\x0c\x??

Ejercicio 5

* Construir a mano un paquete de HCI, más concretamente el paquete de reset

* Core 6.2, Volume 4, Part E, Section 5.4.1

Parameter length

|

v

\x01\x03\x0c\x00

Ejercicio 5

- * Construir a mano un paquete de HCI, más concretamente el paquete de reset

- * Core 6.2, Volume 4, Part E, Section 5.4.1

Reset Command:

\x01\x03\x0c\x00

Ejercicio 5

* Enviar el paquete de reset a nuestro Controller

<https://usbbluetooth.github.io/quirks/>

* Recibir la respuesta...

Ejercicio 5

* Enviar el paquete de reset a nuestro Controller

<https://usbbluetooth.github.io/quirks/>

* Recibir la respuesta...

```
c = get_controller()
```

```
c.open()
```

```
...
```

```
...
```

```
c.close()
```

Ejercicio 5

* Enviar el paquete de reset a nuestro Controller

<https://usbbluetooth.github.io/quirks/>

* Recibir la respuesta...

```
c = get_controller()
```

```
c.open()
```

```
c.write(b"\x01\x03\x0c\x00")
```

```
print(c.read())
```

```
c.close()
```

Ejercicio 6

* Hemos recibido la respuesta al comando, ¿Qué nos dice?

Core 6.2, Volume 4, Part E, Section 5.4.4

“\x04\x0f\x04\x00\x01\x00\x00”

Ejercicio 6

* Hemos recibido la respuesta al comando, ¿Qué nos dice?

Core 6.2, Volume 4, Part E, Section 5.4.4

Event packet

|
v

“\x04\x0f\x04\x00\x01\x00\x00”

Ejercicio 6

* Hemos recibido la respuesta al comando, ¿Qué nos dice?

Core 6.2, Volume 4, Part E, Section 5.4.4

Event code (Command Status)

|
v

"\x04\x0f\x04\x00\x01\x00\x00"

Ejercicio 6

* Hemos recibido la respuesta al comando, ¿Qué nos dice?

Core 6.2, Volume 4, Part E, Section 5.4.4

Parameter length (4 bytes)

|
v

"\x04\x0f\x04\x00\x01\x00\x00"

Ejercicio 6

* Hemos recibido la respuesta al comando, ¿Qué nos dice?

Core 6.2, Volume 4, Part E, Section 5.4.4

Param: Status (OK!)

|
v

"\x04\x0f\x04\x00\x01\x00\x00"

Ejercicio 6

* Hemos recibido la respuesta al comando, ¿Qué nos dice?

Core 6.2, Volume 4, Part E, Section 5.4.4

Param: Num of allowed commands (1)

|
v

“\x04\x0f\x04\x00\x01\x00\x00”

Ejercicio 6

* Hemos recibido la respuesta al comando, ¿Qué nos dice?

Core 6.2, Volume 4, Part E, Section 5.4.4

Param: Opcode (None?)

|
v

"\x04\x0f\x04\x00\x01\x00\x00"

Ejercicio 7

* Resetear nuestro controller pero ahora con scapy-usbbluetooth

Ejercicio 7

* Resetear nuestro controller pero ahora con scapy-usbbluetooth

```
import usbbluetooth
```

```
from scapy_usbbluetooth import UsbBluetoothSocket
```

```
from scapy.all import *
```

```
def get_controller():
```

```
    devices = usbbluetooth.list_controllers()
```

```
    return devices[0]
```

Ejercicio 7

* Resetear nuestro controller pero ahora con scapy-usbbluetooth

```
socket = UsbBluetoothSocket(get_controller())  
pkt = HCI_Hdr() / HCI_Command_Hdr() / HCI_Cmd_Reset()  
response = socket.sr1(pkt)  
response.show()
```

Ejercicio 8

- * Volcar el tráfico del socket a un archivo (scapy-socketlogger)
- * Analizar el archivo PCAP con Wireshark

Ejercicio 8

- * Volcar el tráfico del socket a un archivo (scapy-socketlogger)
- * Analizar el archivo PCAP con Wireshark

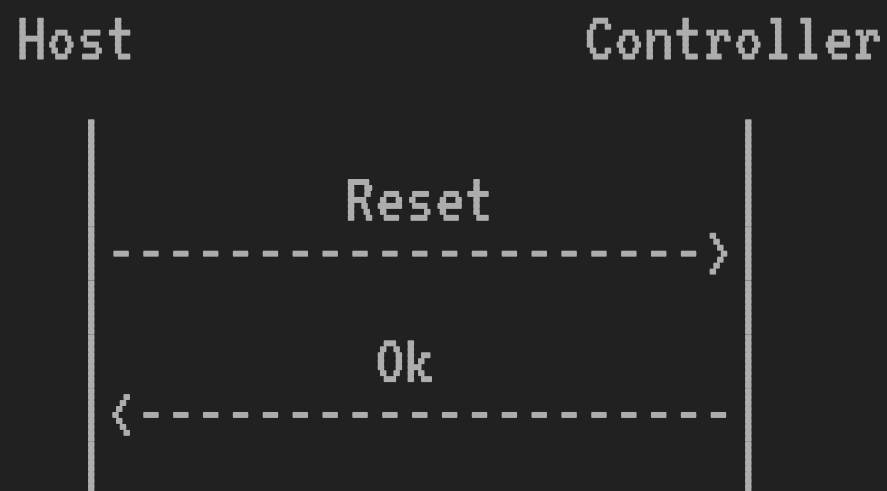
```
pcap_writer = PcapWriter("bluetooth.pcap")  
logger = SocketLogger(socket, pcap_writer)  
...  
logger.close()
```

Inicialización de un controller

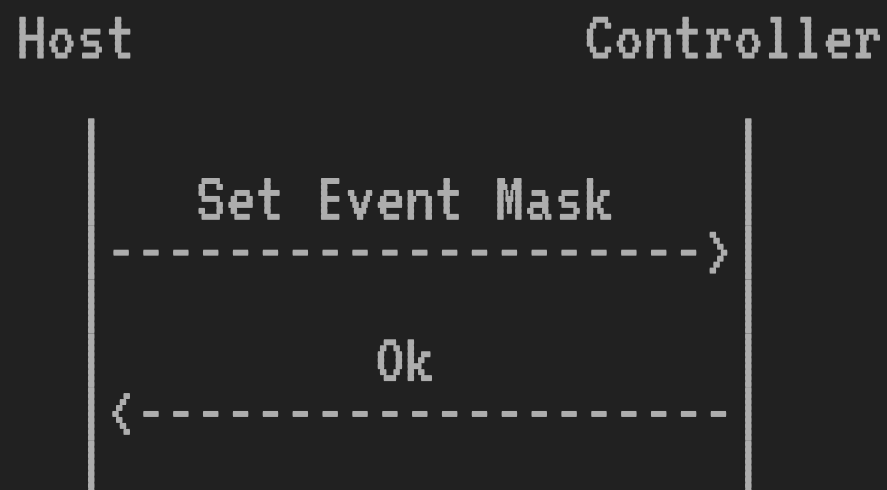
¿Para qué sirve?

- * El controller tiene un “estado”
- * Cuando tomamos el control de un controller no sabemos en que “estado” está
- * Core 6.2, Volume 6, Part D, Section 2.1

Paso 1



Paso 2



Ejercicio 9

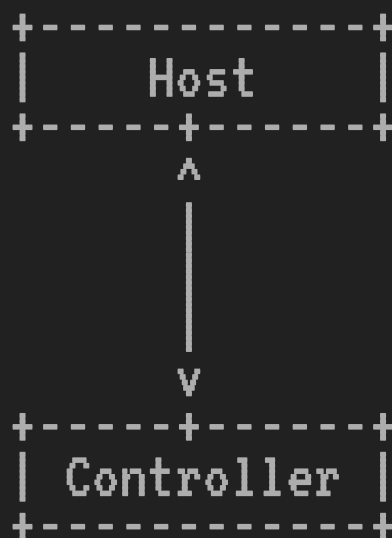
- * Enviar el comando "Set Event Mask" con los parámetros por defecto de Scapy
- * Verificar la respuesta del Controller

Resumen

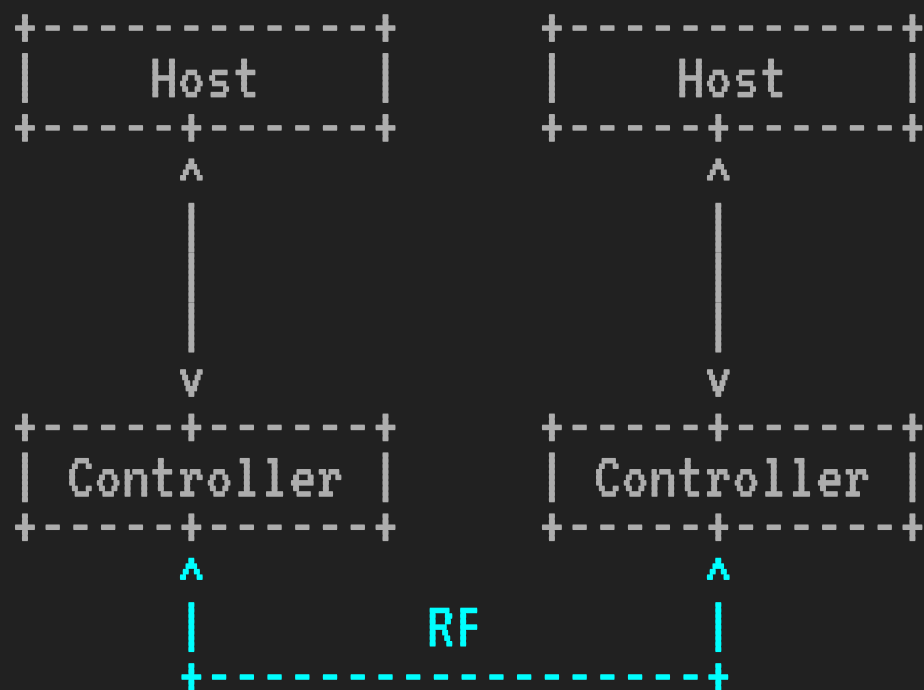
- * El proceso de inicialización es más complejo
- * Implementamos una pequeña parte que nos es útil para estos ejercicios
- * Es posible que no todos los controllers estén ahora mismo en el mismo estado

Capa física

Arquitectura



Arquitectura



BR/EDR

- * Core 6.2, Volume 2, Part A

- * Modulación: GFSK, $\pi/4$ -DQPSK o 8DPSK

- * Frecuencia: 2.400GHz a 2.480GHz

- * Canales: 79 canales de 1MHz

- * Inquiry: Hopping usando 32 canales (secuencia pública)

- * Paging: Hopping usando 32 canales (secuencia basada en MAC)

- * Conexión: Usa un hopping pseudoaleatorio (basado en MAC)

BLE

- * Core 6.2, Volume 6, Part A

- * Modulación: GFSK

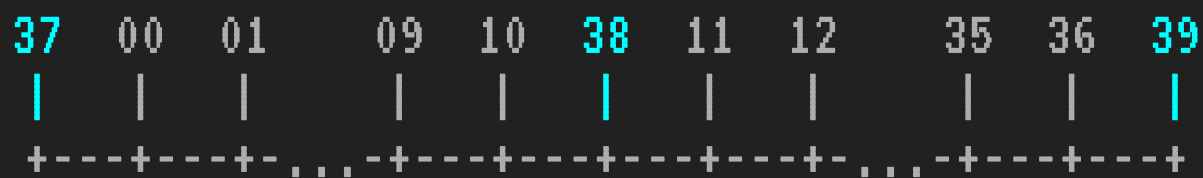
- * Frecuencia: 2.400GHz a 2.480GHz

- * Canales: 40 canales de 2MHz

- * Advertisement: Solo en canales 37, 38, 39

- * Conexión: Usa un hopping pseudoaleatorio (basado en MAC)

Orden de canales en BLE



Channel hopping

- * Consiste en saltar de canal en canal para transmitir paquetes
- * Requiere que el transmisor y el receptor se sincronicen
- * Comunicaciones más resilientes a la interferencia
- * Hace que el sniffing de comunicaciones sea impráctico
 - + Existen proyectos con hardware poco asequible
 - + Otros capturan el inicio de conexión y tratan de seguirla

Ataques usando la capa física

- * Requieren hardware especializado, normalmente “caro”
 - + SDR con TX, Ubertooth...
- * Requieren modificar software o firmware muy sensible a “timings”
- * Los ataques suelen requerir precisión
- * Se requiere “mejor señal” que otros dispositivos en tu alrededor
- * En general son poco prácticos/asequibles

Recibiendo advertisements

Advertisement

- * Es un tipo de paquete Bluetooth
- * Sirve para anunciar la presencia de un dispositivo
 - + Datos imprescindibles para iniciar una conexión
 - + Datos opcionales
- * Se transmite solo en "algunos" canales

Ejercicio 10

- * Enviar al controller los paquetes necesarios para un escaneo LE
- * Guardar los resultados en un PCAP

Ejercicio 10

- * Enviar al controller los paquetes necesarios para un escaneo LE
- * Guardar los resultados en un PCAP

Habilitar escaneo

HCI_Hdr() / HCI_Command_Hdr() /

HCI_Cmd_LE_Set_Scan_Enable(enable=1, filter_dups=0)

Ejercicio 10

- * Enviar al controller los paquetes necesarios para un escaneo LE
- * Guardar los resultados en un PCAP

```
# Recibir los advertisements
```

```
while True:
```

```
    socket.recv().show()
```

Ejercicio 11

- * Analizar el PCAP con Wireshark
- * ¿De qué canal vienen los advertisements?

Canales de advertisement

- * El controller sabe en qué canal ha capturado el advertisement
- * No lo envía al Host
- * Existen proyectos que si lo hacen: Sniffle, nRF sniffer, Ubertooth
- * No nos son útiles por ahora, pero existen

Ejercicio 11

- * Analizar el PCAP con Wireshark
- * ¿Hay algún dispositivo interesante?
- * ¿Qué datos podemos conocer de los dispositivos que nos rodean?

Enviando advertisements

Advertising parameters (Optional)

- * LE Set Advertising Parameters command
- * [Core 6.2, Volume 4, Part E, Section 7.8.5](#)
- * Permite cambiar:
 - + Intervalos de tiempo
 - + Tipo de anuncio
 - + Tipo de MAC a usar
 - + Canales

Advertising data (Optional)

- * LE Set Advertising Data command
- * [Core 6.2, Volume 4, Part E, Section 7.8.7](#)
- * Permite cambiar:
 - + Datos de anuncio
 - + Nombre
 - + UUIDs de servicios
 - + ...

Advertising enable

- * LE Set Advertising Enable command
- * Core 6.2, Volume 4, Part E, Section 7.8.9
- * Permite cambiar:
 - + Si habilitamos anuncios o no

Ejercicio 12

* Implementar los comandos para enviar advertisements con vuestro nombre/nick

Ejercicio 12

* Implementar los comandos para enviar advertisements con vuestro nombre/nick

```
# Set advertising parameters
```

```
HCI_Hdr() / HCI_Command_Hdr() /
```

```
HCI_Cmd_LE_Set_Advertising_Parameters()
```

Ejercicio 12

* Implementar los comandos para enviar advertisements con vuestro nombre/nick

```
# Set advertising data
```

```
HCI_Hdr() / HCI_Command_Hdr() /
```

```
HCI_Cmd_LE_Set_Advertising_Data(data=[
```

```
    EIR_Hdr() / EIR_CompleteLocalName(local_name=b"Anton")
```

```
])
```

Ejercicio 12

* Implementar los comandos para enviar advertisements con vuestro nombre/nick

```
# Set advertising enable
```

```
HCI_Hdr() / HCI_Command_Hdr() /
```

```
HCI_Cmd_LE_Set_Advertise_Enable(enable=1)
```

Ejercicio 13

* ¿Qué ataques podemos realizar con esta funcionalidad?

Ejercicio 13

* ¿Qué ataques podemos realizar con esta funcionalidad?

- + Format string

- + Encoding attacks

- ...

Advertisement Manufacturer Data

- * Es un tipo de advertisement data
- * Está definido por algún fabricante asociado a un ID
- * Normalmente sus especificaciones no son públicas
- * Suele requerir reversing para comprenderlos
- * Más comunes: Apple, Google, Samsung, Microsoft...

Microsoft Manufacturer Data

- * El company ID de microsoft es 0x0006
- * Un tipos de paquete se llama "SwiftPair"
- * El paquete tiene un formato parcialmente conocido:
 "\x03\x00\x80<string nombre>\x00"

Ejercicio 14

- * Craftear un spam attack para Microsoft
- * Manufacturer ID: 0x0006
- * Manufacturer data: “\x03\x00\x80<string nombre>\x00”

Ejercicio 14

- * Crafter un spam attack para Microsoft

- * Manufacturer ID: 0x0006

- * Manufacturer data: “\x03\x00\x80<string nombre>\x00”

```
# Set manufacturer advertising data
```

```
HCI_Hdr() / HCI_Command_Hdr() /
```

```
HCI_Cmd_LE_Set_Advertising_Data(data=[EIR_Hdr() /
```

```
EIR_Manufacturer_Specific_Data(company_id=0x0006) /
```

```
EIR_Raw(data=b"\x03\x00\x80\x41\x41\x41\x41\x41\x00"])]
```

Spam attacks

- * Podéis encontrar más ejemplos de spam attacks en <https://github.com/TarlogicSecurity/BluetoothExamplesAndDemos/tree/main/BleSpammer>
- * Samsung
- * Microsoft
- * Apple

Vendor HCI Commands

Vendor HCI Commands

- * El estándar no pretende cubrir el 100% de los casos de uso
- * Para los casos “custom” permite comandos de vendedor
- * Pueden implementar cualquier tipo de funcionalidad
- * Cambian según nuestro controller

Ejercicio 15

* ¿Qué fabricante y modelo de controller tiene cada uno?

<https://github.com/antoniovazquezblanco/Bluetooth-Controllers-Reversing>

Documentación

* Existen proyectos para documentar estos comandos:

<https://github.com/TarlogicSecurity/BluetoothExamplesAndDemos/blob/main/VendorCommandEnumerator/>

<https://github.com/antoniovazquezblanco/Bluetooth-Controllers-Reversing>

https://github.com/darkmentorllc/BT_Security_VSC_DB

* Los siguientes pasos serán dar soporte en Scapy...

Ejemplo dump de firmware Realtek

```
class HCI_Cmd_VSC_Realtek_Read_Mem(Packet):  
    name = "Realtek Read Mem"  
    fields_desc = [  
        ByteField("size", 0x20),  
        XLEIntField("address", 0x80000000)  
    ]  
  
bind_layers(HCI_Command_Hdr, HCI_Cmd_VSC_Realtek_Read_Mem,  
    ogf=0x3f, ocf=0x0061)
```

Ejemplo dump de firmware Realtek

```
class HCI_Cmd_Complete_VSC_Realtek_Read_Mem(Packet):  
    name = 'Realtek Read Mem complete'  
    fields_desc = [  
        XStrLenField("data", 0, length_from=lambda pkt:  
pkt.underlayer.underlayer.len)  
    ]  
  
bind_layers(HCI_Event_Command_Complete,  
HCI_Cmd_Complete_VSC_Realtek_Read_Mem, opcode=0xfc61)
```

Ejemplo dump de firmware Realtek

```
def read(socket, address=0x80000000):  
    pkt = HCI_Hdr() / HCI_Command_Hdr() /  
    HCI_Cmd_VSC_Realtek_Read_Mem(address=address)  
  
    response = socket.sr1(pkt, verbose=0)  
  
    if not HCI_Event_Command_Complete in response or  
    response[HCI_Event_Command_Complete].status != 0:  
        return None  
  
    return response.data
```

Ejemplo dump de firmware Realtek

```
for addr in range(begin_addr, end_addr, 4):  
    print(f"[+] Reading @ 0x{addr:08x}")  
    data = read(socket, addr)  
  
    if data is None:  
        print("[!] Error!")  
        break  
  
    f.write(data)
```

Ejemplo dump de firmware Realtek

* Con este firmware podemos realizar ingeniería inversa del controller...

* Xeno Kovah está construyendo un firmware custom con "capacidades avanzadas"

https://github.com/darkmentor11c/DarkFirmware_real_i

https://darkmentor.com/2025-11-21_HardwarioNL2025_RTL8761B_RE_Slides_With_Builds.pdf

Comandos comunes

- * Cambiar la MAC del dispositivo
- * Leer y escribir la RAM / ROM del controlador
- * Cambiar potencias de transmisión
- * Enviar y recibir paquetes de bajo nivel!

Ejercicio 16

* ¿Qué podemos hacer con estos comandos?

Ejercicio 16

- * ¿Qué podemos hacer con estos comandos?

- * Esconder nuestro código en dongles

- + Permitir conexiones de una MAC concreta siempre

- + Exfiltrar datos/keystrokes

- * Forzar que todas las conexiones sean sin hopping/cifrado

- * Suplantar MACs/dispositivos!

Reflexiones

- * Parte del modelo de seguridad de Bluetooth es que los Controllers no pueden realizar ciertos tipos de operaciones
- * El estándar de Bluetooth añade vendor commands no supervisados
- * Los fabricantes implementan lectura y escritura de código
- * Bluetooth no es seguro desde este punto de vista...

Bluetooth Security Assessment Methodology (BSAM)

BSAM

- * Trata de ser una guía completa para la seguridad de Bluetooth
- * Es un conjunto de comprobaciones a realizar para asegurar un dispositivo Bluetooth
- * Está pensada desde el punto de vista “defensivo”
- * Es una buena manera de entender las vulnerabilidades más comunes

BSAM

- * También documenta técnicas que permiten implementar o testear alguno de los controles
- * Estas técnicas son útiles para depurar nuestras PoCs
- * Estas técnicas también pueden ser útiles para implementar ataques

BSAM

* Podéis encontrar la metodología en:

<https://www.tarlogic.com/bsam/es/>

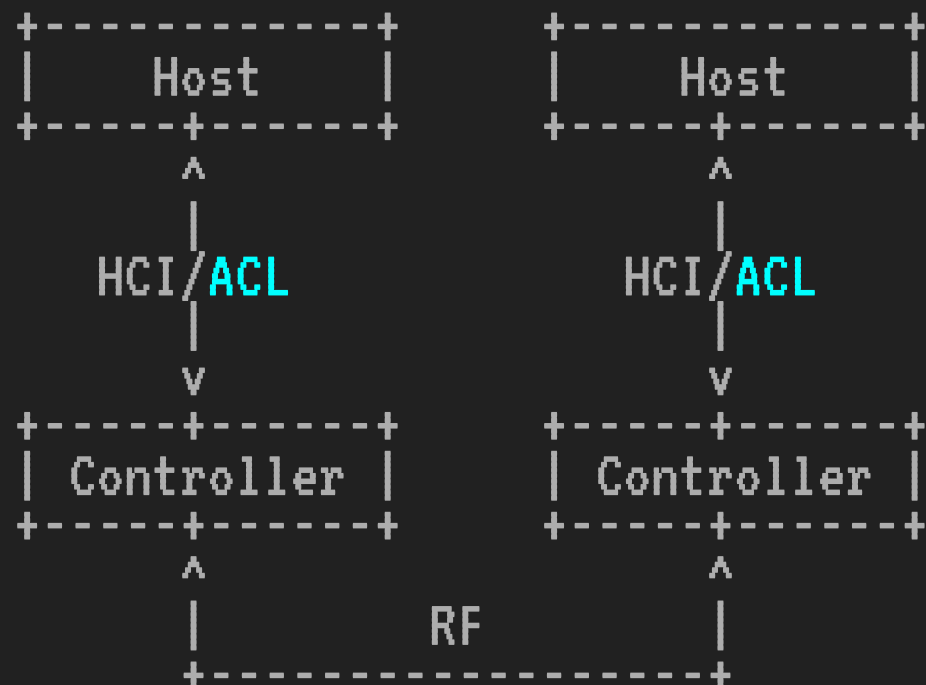
<https://github.com/TarlogicSecurity/BSAM>

Conexión

Conexión

- * Llamamos conexión a crear un canal de conexión Host a Host
 - + Le decimos al controller que se conecte
 - + El controller se encarga de mantener un enlace RF
 - + Usamos ACL como un túnel entre hosts
- * Solo se puede establecer conexión si el dispositivo está en estado conectable!

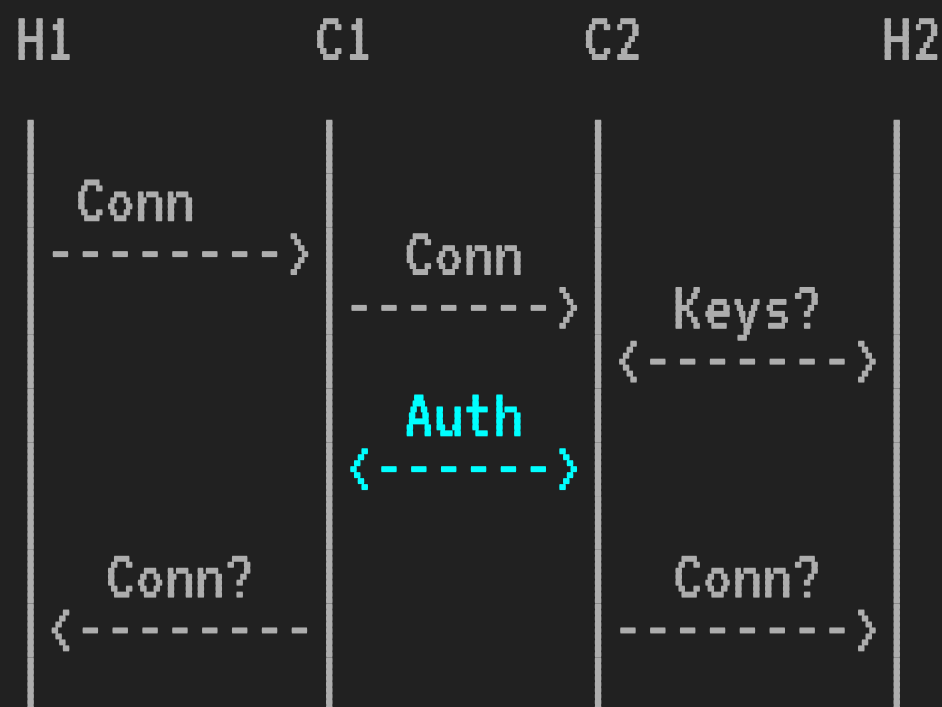
Conexión



Conexión

```
+-----+ ACL +-----+
| Host | <---> | Host |
+-----+     +-----+
```

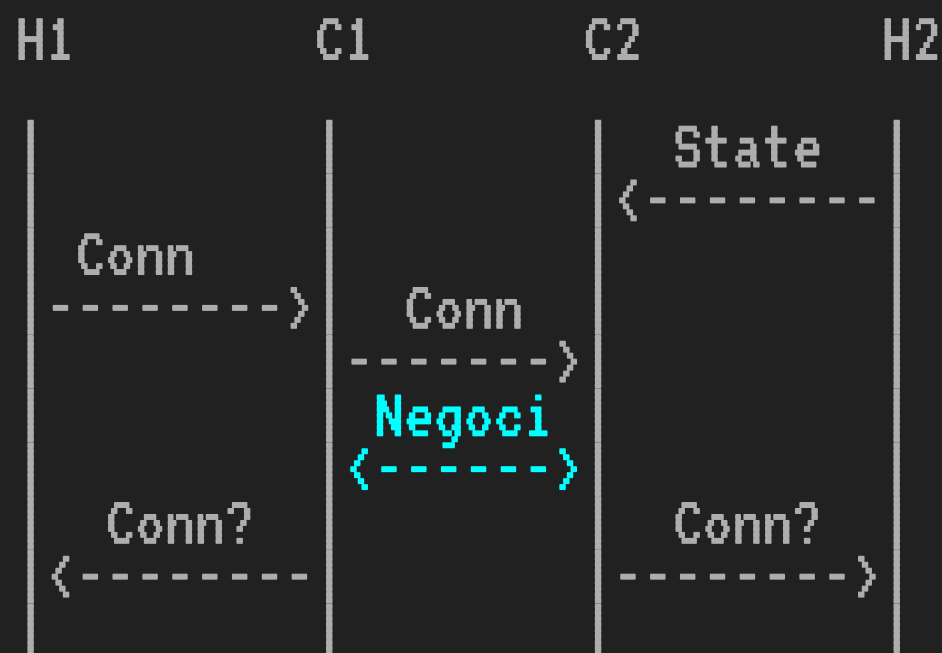
Establecimiento de conexión BR/EDR



Establecimiento de conexión BR/EDR

- * La mayor parte del proceso transcurre entre los controllers
- * Según el std, requiere el proceso de autenticación
- * Si dos dispositivos no se conocen, requiere pairing
- * “No es posible conexión entre dos dispositivos sin autenticación o bypass de autenticación”
- * En la siguiente sección veremos el proceso de autenticación

Establecimiento de conexión LE



Establecimiento de conexión LE

- * La conexión es independiente de la autenticación
- * Se puede establecer conexión e intercambiar información
- * Existe un control de acceso a nivel de host, no a nivel de controller

Herramientas

- * Hasta ahora hemos interactuado al más bajo nivel
- * Implementar todo un cliente de Bluetooth es complejo
- * Usaremos **Bleak** para nuestras pruebas

Ejercicio 17

- * Completar el script de ejemplo para que:
 - + Se conecte a un dispositivo LE

Ejercicio 17

* Completar el script de ejemplo para que:

- + Se conecte a un dispositivo LE

```
# Connection
```

```
console.log(f"Connecting to {target}...")
```

```
async with BleakClient(target) as client:
```

```
    console.log(f"Connected to {target}!")
```

Pairing y autenticación

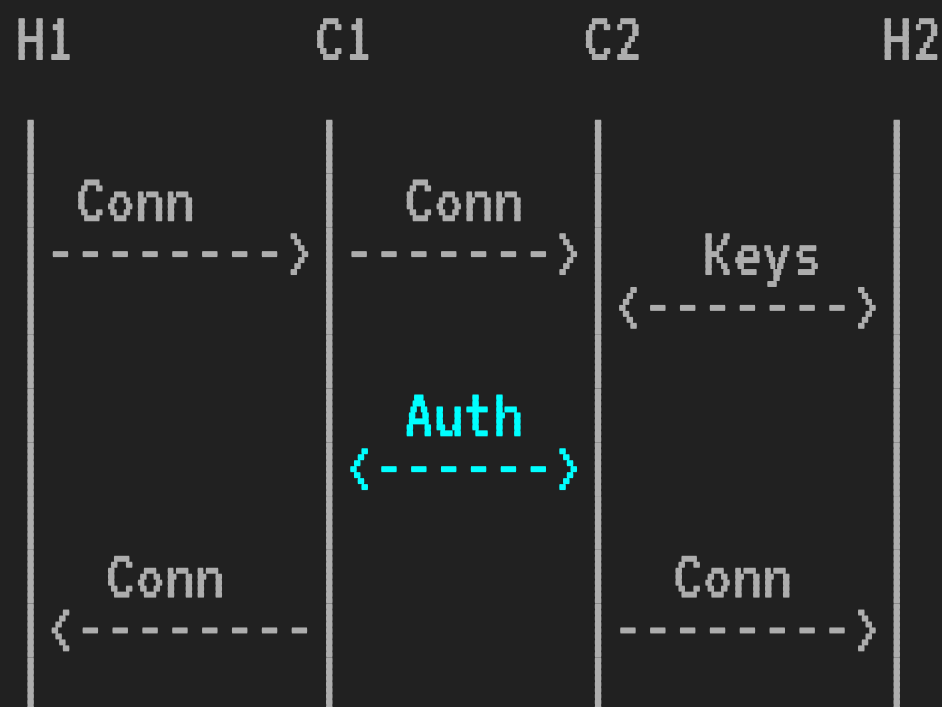
Autenticación

- * Consiste en validar la identidad de un dispositivo
- * Se valida una MAC pública (Esa es la identidad)
- * Se valida mediante una LTK (Long Term Key)
- * Una LTK es válida para una pareja de MACs

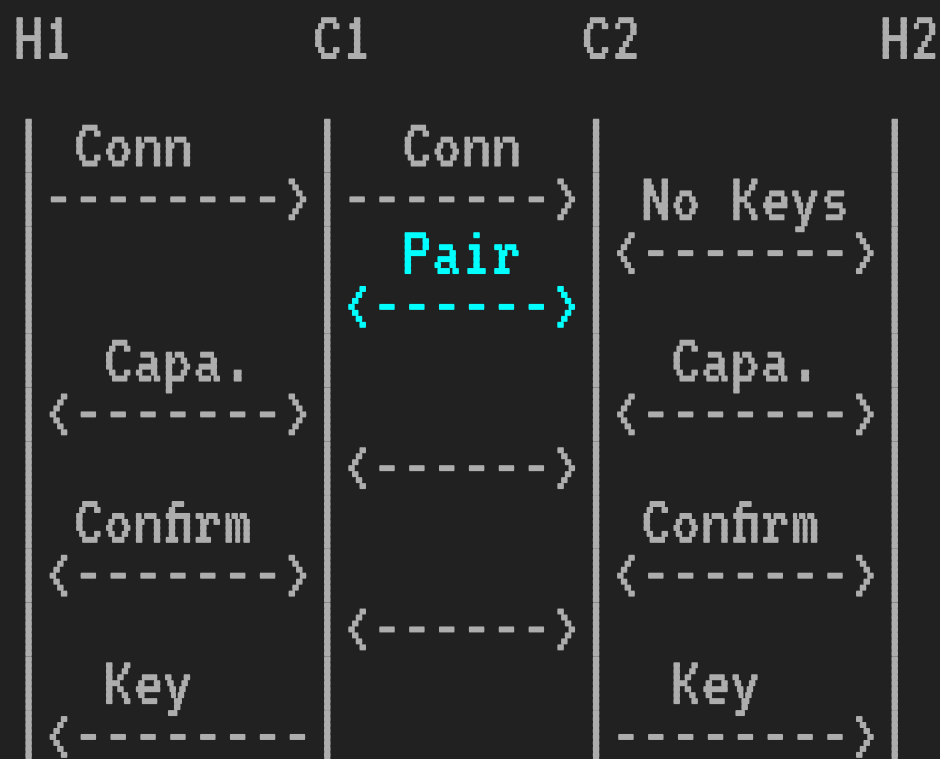
Pairing

- * Burdamente: Es una autenticación manual...
- * La primera vez que conectamos 2 dispositivos no tenemos LTK...
- * Podríamos introducir la clave en los 2 manualmente...
 - + Todas las claves serían malísimas...
 - + No todos los dispositivos tienen teclado...
- * Se usa criptografía y se invita al usuario a comprobar mediante pin, confirmación visual u otros...

Autenticación BR/EDR



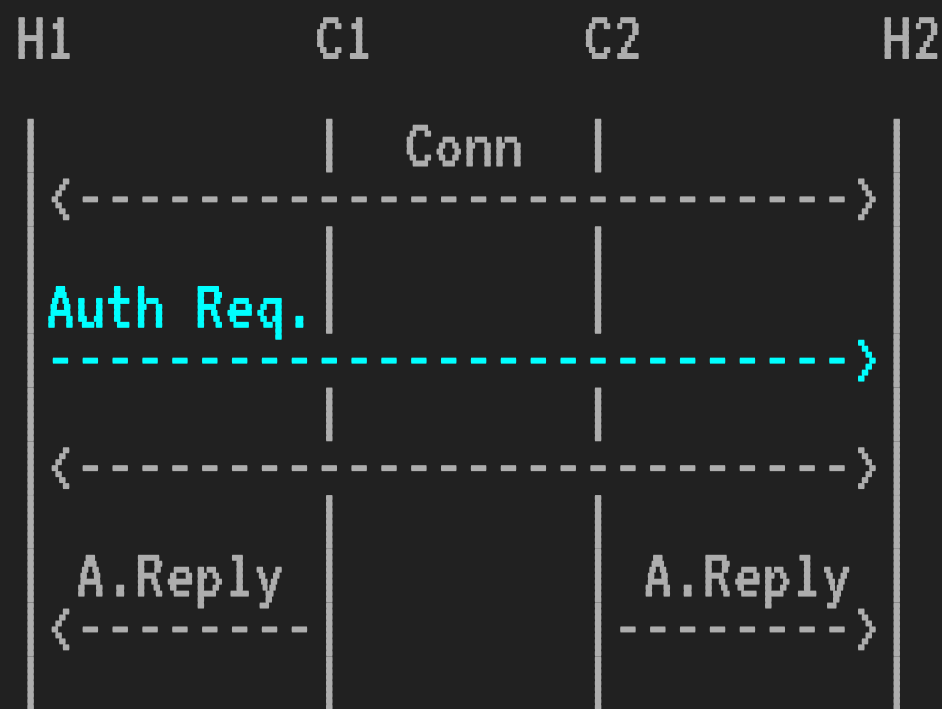
Pairing BR/EDR



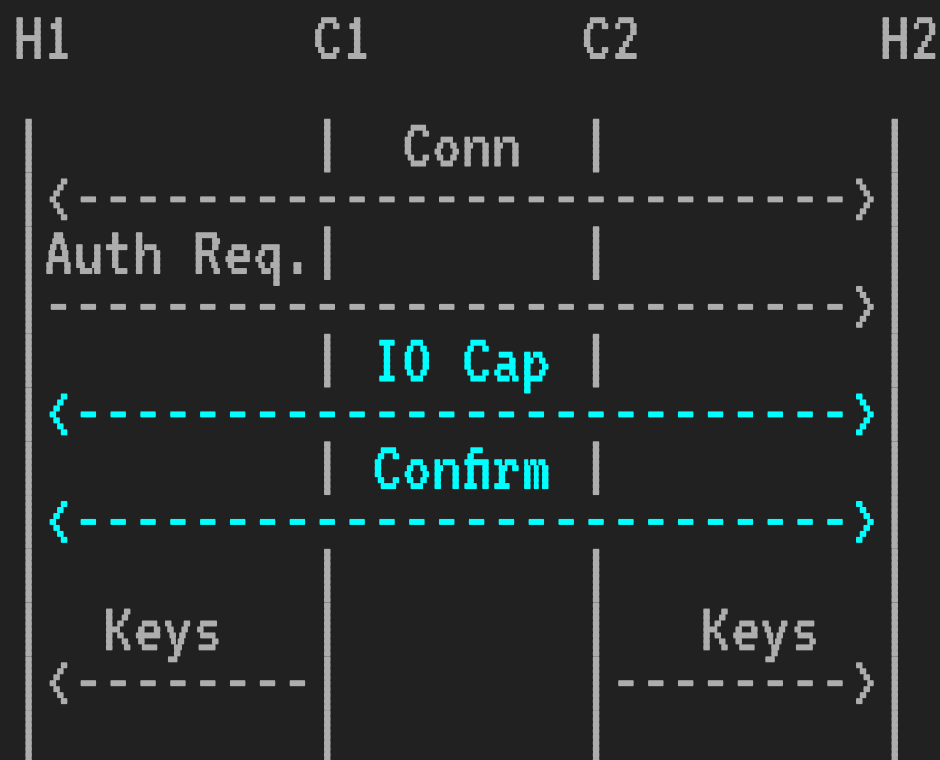
Pairing BR/EDR

- * Core 6.2, Volume 2, Part F, Section 3.1
- * La mayor parte del proceso transcurre entre los controllers
- * Existen 2 versiones según la criptografía: Legacy o Secure
- * Durante el proceso, desde el host tenemos control sobre:
 - + Si tenemos keys para un dispositivo o no
 - + Los “capabilities” de nuestro dispositivo
 - + La confirmación final del pairing

Autenticación LE



Pairing LE



Pairing LE

- * Similar a Classic pero a “nivel de host”
- * Se usa el protocolo SMP (Security Manager Protocol)
- * Durante el proceso, desde el host tenemos control sobre:
 - + Si tenemos keys para un dispositivo o no
 - + Los “capabilities” de nuestro dispositivo
 - + La confirmación final del pairing

Ejercicio 18

- * Capturar un emparejamiento con un dispositivo...
 - + Podemos usar "btmon" y "bluetoothctl" en Linux
 - + Podemos usar Bleak en cualquier plataforma
- * Ojear el proceso en Wireshark...

Capabilities

- * Core 6.2, Volume 3, Part H, Section 2.3.5.1, Table 2.8
- * Un dispositivo puede tener las siguientes “IO Capabilities”:
DispOnly, DispYesNo, KeybOnly, NoInputNoOutput, KeybDisp
- * Se negocia la confirmación en base a “IO Caps” de ambos...

Ataques pairing y autenticación BR/EDR

- * Downgrade de capabilities!

- + Tenemos control sobre los “capabilities” que mostramos
- + Usamos procesos estándar! Cualquier tool nos vale!
- + Si nos mostramos como “NoInputNoOutput”...

- * El resto de ataques requieren acceso a los paquetes RF o modificaciones del sw del controller...

Ejercicio 19

- * Setear nuestros "IO Capabilities" a "NoInputNoOutput"...
- * Capturar un emparejamiento con un dispositivo...
 - + Podemos usar "btmon" y "bluetoothctl" en Linux
 - + Podemos usar Bleak en cualquier plataforma
- * Ojear el proceso en Wireshark...

Ejemplo

- * BlueSpy
- * Se conecta a dispositivos haciendo downgrades
- * Captura audio de estos dispositivos
- * <https://github.com/TarlogicSecurity/BlueSpy>

Servicios

Servicios

- * Bluetooth nos ofrece protocolos de transporte para conectarnos Host a Host: ACL y L2CAP
- * Por encima de los protocolos de transporte, tenemos “servicios”
- * Los servicios son perfiles que dan una funcionalidad conocida
 - + Existen muchos estandarizados por el SIG...
 - + Otros definidos por fabricante y documentados o no...

Servicios BR/EDR

- * Service Discovery Protocol (SDP) para descubrir servicios/perfiles
- * Core 6.2, Volume 3, Part B
- * Cada perfil tiene su definición, protocolos y canal de L2CAP
- * Bluetooth Assigned Numbers, Section 6

Servicios LE

- * Generic Attribute Profile (GATT) para descubrir protocolos
- * Core 6.2, Volume 3, Part G
- * Attribute Protocol (ATT) como mecanismo estándar para servicios
- * Core 6.2, Volume 3, Part F

Ataques a descubrimiento de servicios

- * Hay mecanismos de descubrimiento de servicios
- * Pueden no estar bien configurados...
- * Pueden ocultar servicios a proposito...
- * Se puede hacer fuerza bruta de servicios!

Ejercicio 20

* Completar el script de conexión de Bleak para listar los servicios de ese dispositivo

Ejercicio 20

* Completar el script de conexión de Bleak listar los servicios de ese dispositivo

```
# Services
```

```
for service in client.services:
```

```
    console.log(f"[Service] {service}")
```

Estructura de servicios ATT

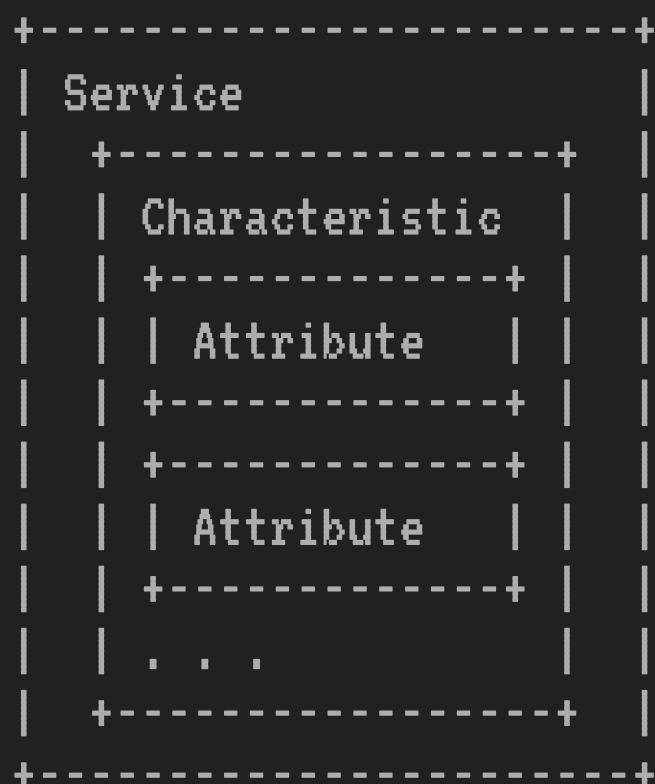
+-----+		
	Service	
	+-----+	
	Characteristic	
	+-----+	
	+-----+	
	Characteristic	
	+-----+	
	. . .	
+-----+		

Estructura de servicios ATT

Ejemplo:

- * Un reloj deportivo Bluetooth LE tiene múltiples servicios
 - + Service: pulsioximetría
 - Characteristic: Pulso
 - Characteristic: Saturación de oxígeno

Estructura de servicios ATT



Estructura de servicios ATT

Ejemplo:

* Un reloj deportivo Bluetooth LE tiene múltiples servicios

- + Service: pulsioximetría

- Characteristic: Pulso

- Attribute: Ritmo cardíaco

-

- Characteristic: Saturación de oxígeno

- Attribute: Concentración de oxígeno

- Attribute: Configuración de precisión

Ejercicio 21

* Completar el script de conexión de Bleak listar los characteristics de ese dispositivo

Ejercicio 21

* Completar el script de conexión de Bleak listar los characteristics de ese dispositivo

```
# Characteristics
```

```
for char in service.characteristics:
```

```
    console.log(f" [Characteristic] {char}")
```

Ataques a servicios

- * Puede que haya servicios o características que no deberían ser accesibles y sí lo son...
 - + Se debería probar mediante fuerza bruta (lectura y escritura)
 - + Se puede combinar con bypass de auth
- * Puede que haya servicios mal implementados!
 - + Debemos pensar cómo se han implementado...
 - + Se procesan bien los inputs?

Ejercicio 22

- * Hay algún servicio en el que podamos leer y/o escribir?
- * Leer alguno de los atributos de nuestro dispositivo...

Ejercicio 22

- * Hay algún servicio en el que podamos leer y/o escribir?
- * Leer alguno de los atributos de nuestro dispositivo...

```
# Read characteristic
```

```
char_uuid = "deadbeef-dead-beef-dead-beefdeadbeef"
```

```
console.log(await client.read_gatt_char(char_uuid))
```


Ejercicio 23

* ¿Es seguro este servicio?

Ejercicio 23

- * ¿Es seguro este servicio?
- * ¿Se sanean bien los inputs?

Ejercicio 23

- * ¿Es seguro este servicio?
- * ¿Se sanean bien los inputs?
- * ¿Cual es el flag?

Chaos

Chaos

- * Vamos a tratar de vulnerar cualquier dispositivo que hayamos traído

- * Descubrir que chip lleva nuestro Dongle

- * Descubrir comandos de vendedor de nuestro Dongle

- * Tratar de saltar la autenticación de nuestros dispositivos

- * Tratar de vulnerar algún servicio de nuestros dispositivos

- ...

Chaos

- * Cambio de MAC pública para nuestro dispositivo (Suplantación)
- * Montar un escáner de dispositivos (Con logging GPS?)
- * Dumpear firmware de nuestros dongles
- ...